QUANTUMSHIELD AI

Project Explanation + Revised 28-Day Roadmap

Beginner-friendly Hinglish guide

Project vision: AI-assisted digital forensics, explainable incident investigation, and post-quantum evidence integrity.

Who this is for: A complete beginner building a safe, demo-ready student prototype in manageable stages.

Important: 28 days is a structured learning target, not a guarantee of a production-grade forensic platform. Use synthetic data and authorised lab environments.

Part 1 — Project Explanation

1. Problem statement: kya problem solve kar rahe hain?

Real cyber incidents mein investigators ko authentication logs, web logs, process records, network events aur suspicious files ke beech connections dhoondhne padte hain. Data bahut zyada ho sakta hai, formats alag hote hain, aur manually incident sequence banana time-consuming hota hai.

QuantumShield AI ka goal: evidence ko organise karna, suspicious activity flag karna, possible incident timeline banana, AI ke through evidence-linked explanation dena, aur evidence manifest ki integrity verify karne mein help karna.

Ye human investigator ko replace nahi karta. Iska output investigation lead hai; final conclusion ko human review aur source evidence se verify karna hoga.

2. Example scenario

Ek synthetic case mein employee ko suspicious email milta hai, file download hoti hai, unknown process execute hota hai, unusual outbound connection record hota hai, aur repeated login failures milte hain. System timestamps aur source records ko organise karke possible sequence highlight karega.

Time	Synthetic event	Investigation value
10:00	Suspicious email received	Possible initial clue
10:03	File downloaded	File metadata / source to inspect
10:04	Unknown process executed	Process details to verify
10:05	Unusual outbound connection	Destination and process correlation
10:07	Repeated failed logins	Authentication context to inspect

Important: chronological order alone does not prove causation. The system should label this as a possible sequence and show the supporting event IDs.

3. Main modules

A. Evidence Collector

CSV/JSON synthetic logs import karega; metadata, source name aur timestamps record karega. First prototype mein prebuilt data safest aur easiest option hai.

B. Detection Engine

Python rules known patterns identify karenge, jaise configurable time window mein repeated failed logins. Optional machine learning unusual events flag karega, lekin anomaly attack ka proof nahi hoti.

C. Timeline Engine

Normalised events ko timestamp ke according sort karega, related event IDs ko group karega, aur missing data / uncertainty show karega.

D. AI Investigation Assistant

LLM + simple retrieval/RAG relevant records select karke natural-language questions answer karega. Answers mein evidence IDs, uncertainty aur recommended next checks honge. Unsupported claims ko avoid karna aur 'evidence insufficient' kehna zaroori hai.

E. Evidence Integrity

SHA-256 file-content fingerprint deta hai. Signed manifest mein case ID, filenames, hashes, timestamps aur tool version store ho sakte hain. ML-DSA signature sign/verify kar sakti hai. Hash match se source ki truthfulness ya legal admissibility prove nahi hoti.

F. Reporting

Streamlit dashboard par alerts, event timeline, evidence details, AI explanation aur verification status dikhaye jayenge. ReportLab PDF export generate karega.

4. Architecture / data flow

- 1. Synthetic CSV/JSON evidence input
- 2. Validate files; record metadata; calculate SHA-256
- 3. Parse and normalise events into a common schema
- 4. Run detection rules and create alerts
- 5. Sort events and create a possible incident timeline
- 6. Retrieve relevant evidence for AI questions and cite event IDs
- 7. Sign/verify evidence manifest where PQC implementation is available
- 8. Display dashboard and export report

5. What the prototype can and cannot do

Can do in the prototype	Cannot claim to do
Analyse synthetic CSV/JSON logs	Detect every real-world attack
Flag suspicious patterns using defined rules	Treat every alert as a confirmed attack
Build a timestamp-based possible timeline	Recover missing logs or prove causation automatically
Generate evidence-linked AI explanations	Guarantee no AI hallucinations
Compare SHA-256 hashes and verify signatures	Prove evidence was truthful or legally admissible
Demonstrate ML-DSA signed-manifest verification	Claim the whole application is quantum-proof
Suggest investigation next steps	Identify a criminal's real-world identity automatically

6. Recommended technology stack

Layer	Suggested tools
Language	Python 3; basic SQL
Interface	Streamlit; Plotly for timeline/charts
Data handling	csv, json, datetime, re, pathlib, pandas, Pydantic
Database	SQLite initially; PostgreSQL only if scale/multi-user needs arise
Detection	Python rules first; scikit-learn / Isolation Forest optional
AI	LLM API or local model; evidence retrieval first; LlamaIndex or LangChain optional
Cryptography	hashlib SHA-256; cryptography library for conventional signature learning; ML-DSA via a supported, verified implementation such as liboqs where compatible
Reports	ReportLab PDF; CSV/JSON exports
Quality	Git/GitHub, venv, pytest, Ruff; Docker optional

7. Suggested repository structure

```
quantumshield-ai/
■■■ app.py
■■■ requirements.txt
■■■ README.md
■■■ data/
■ ■■■ synthetic_case/
■■■ src/
■ ■■■ log_parser.py
■ ■■■ detector.py
■ ■■■ timeline.py
■ ■■■ evidence_hash.py
■ ■■■ signature_manager.py
■ ■■■ evidence_vault.py
■ ■■■ investigator_ai.py
■ ■■■ report_generator.py
■■■ tests/
■ ■■■ test_parser.py
■ ■■■ test_detector.py
■ ■■■ test_integrity.py
■■■ reports/
```

Part 2 — Revised 28-Day Roadmap

Suggested daily target: around 2.5–3 hours. A practical split is 40 minutes learning, 70 minutes coding, 20 minutes testing, 10 minutes notes, plus 10–40 minutes for setup/debugging. If a day takes longer, use a buffer rather than rushing.

WEEK 1 — Python and computer fundamentals

Day 1 — Python basics

- Learn: Python purpose; install Python and VS Code; print(), variables, strings, integers, floats, booleans, input(), type conversion, operators, if/else.
- Practice: print your name and project; check even/odd; create a failed-login threshold alert.
- Example: failed_attempts = 6; if failed_attempts >= 5: print('Suspicious login activity').
- Deliverable: a small threshold-check script.

Day 2 — Loops, functions and collections

- Learn: for/while loops, functions, arguments/return, lists, dictionaries, len(), split(), strip(), lower().
- Practice: count repeated IPs in a list and report total attempts, unique IPs, and most frequent IP.
- Deliverable: IP counter using a dictionary.

Day 3 — Files, CSV, JSON, timestamps and regex

- Learn: open/read, CSV rows, JSON objects, datetime parsing, regex basics.
- Practice: parse lines such as '2026-10-29 10:02:11 FAILED admin 45.33.12.9' into timestamp, status, username, IP.
- Deliverable: a parser that converts one raw line into a structured dictionary; report malformed lines.

Day 4 — Pandas, venv and Git

- Learn: pip, virtual environments, DataFrame, filtering, groupby, Git add/commit/push.
- Practice: count failed logins by IP in a table; create repository and commit.
- Deliverable: first GitHub repository and requirements.txt.

Day 5 — Linux terminal

- Learn: pwd, ls, cd, mkdir, cp, mv, cat, head, tail, grep, permissions and chmod.
- Practice: find FAILED records in your own synthetic log and compare with Python output.
- Deliverable: a short command cheat sheet and filtered log output.

Day 6 — Networking fundamentals

- Learn: IP/MAC, ports, TCP/UDP, DNS, HTTP GET/POST, status codes 200/401/403/404/500, client-server.
- Practice: analyse synthetic access logs and distinguish successful vs failed requests.
- Deliverable: written explanation of what IP, port and status codes tell an investigator.

Day 7 — Buffer + Mini Project 1

- Build: Python Authentication Log Analyzer.
- Features: read .log file, count failed/successful logins, unique IPs, configurable threshold, suspicious-IP report, malformed-line handling.
- Test: normal file, empty file, invalid lines, zero failures, repeated IPs.
- Deliverable: log_analyzer.py, sample synthetic logs, README.

WEEK 2 — Cybersecurity detection and timeline

Day 8 — Cybersecurity fundamentals

- Learn: CIA triad, threat/vulnerability/risk/control, phishing, brute force, malware, incident response, MITRE ATT&CK; overview.
- Practice: for 10 synthetic events, describe what happened, possible risk, available evidence and missing information.
- Deliverable: security-event glossary.

Day 9 — Web attacks and logs

- Learn: brute force, SQL injection concept, XSS concept, phishing, suspicious URL patterns and authentication failures.
- Practice: identify suspicious patterns in prewritten synthetic access logs; do not attack real websites.
- Deliverable: three simple log-based detection rules.

Day 10 — Log types and normalisation

- Learn: authentication logs, web access logs, Windows event-log concepts, process/network records, event ID, timestamp, source, severity.
- Practice: normalise each event into event_id, timestamp, source, event_type, src_ip, username where available.
- Deliverable: parser for supported synthetic log formats.

Day 11 — Safe lab and test events

- Learn: VM vs container, localhost, isolated lab, logs and synthetic vs real evidence.
- Practice: generate synthetic login-failure and suspicious-request events. Optional DVWA only in an isolated, authorised lab and never exposed publicly.
- Deliverable: documented test scenario and expected log output.

Day 12 — Detection rules

- Learn: rule-based detection, thresholds, time windows, false positives/negatives, severity, Sigma overview.
- Practice: repeated failed logins; suspicious web-request patterns; unknown process event. Include alert_id, rule_name, severity, event_ids, reason.
- Deliverable: rule-based detector.

Day 13 — Timeline and correlation

- Learn: timestamp sorting, event correlation, source references, MITRE ATT&CK; tactics, missing logs and clock drift.
- Practice: create a possible sequence such as email → download → process → network connection. Do not claim causation without supporting evidence.
- Deliverable: timeline generator with event IDs and uncertainty notes.

Day 14 — Buffer + Mini Project 2

- Build: Incident Timeline Engine from synthetic auth, web, process and network records.
- Output: normalised events, alerts, chronological timeline, evidence references and a basic incident summary.
- Acceptance test: changing a timestamp updates order; alerts show relevant evidence IDs.
- Deliverable: backend v1.

WEEK 3 — Evidence Vault, cryptography and PQC

Day 15 — SHA-256 hashing

- Learn: hash functions, SHA-256, hash vs encryption, content changes and integrity limitations.
- Practice: hash a sample log file, modify a copy and compare hashes using Python hashlib.
- Deliverable: file hashing utility.

Day 16 — Encryption and digital signatures

- Learn: symmetric/asymmetric cryptography, public/private keys, signing and verification, certificates and trust.
- Practice: use Python cryptography library for a conventional digital signature sign/verify example.
- Deliverable: signed manifest example. Signature is not the same as encryption.

Day 17 — Hash chains and tampering

- Learn: append-only records, previous-record hash, chain verification and trusted checkpoints.
- Practice: link synthetic audit events by previous hash; modify an old event and detect chain failure.
- Deliverable: hash-chain demo. Note: a fully rewritable chain alone is not tamper-proof.

Day 18 — Quantum and PQC concepts

- Learn: high-level quantum computing, Shor’s algorithm impact, post-quantum cryptography, NIST ML-DSA, ML-KEM and SLH-DSA.
- Practice: write a short comparison of conventional signatures and ML-DSA.
- Deliverable: PQC architecture note; no advanced quantum maths required.

Day 19 — ML-DSA setup

- Learn: PQC libraries, version compatibility, key generation, signing and verification.
- Practice: use a supported Open Quantum Safe implementation where compatible. Verify original message, modified message and wrong-key cases.
- Fallback: if installation blocks progress, finish the interface/tests and continue the conventional signature path; do not fake PQC verification or claim it is implemented.

Day 20 — Evidence Vault

- Learn: SQLite tables/queries, evidence metadata, manifest, signature status, read-only originals and working copies.
- Practice: store cases/events/metadata; calculate evidence hashes; create manifest.
- Deliverable: Evidence Vault v1.

Day 21 — Buffer + integration

- Integrate: detector + timeline + evidence vault.
- Test: changed evidence gives hash mismatch; changed signed manifest fails signature verification; malformed logs handled; alerts link to event IDs.
- Deliverable: backend v1 integrated.

WEEK 4 — AI, dashboard and demo

Day 22 — Machine-learning basics

- Learn: rules vs ML, supervised/unsupervised learning, features, model outputs, false positives and Isolation Forest concept.
- Practice: run a small scikit-learn example on synthetic numeric activity data.
- Deliverable: introductory anomaly-detection script/notebook.

Day 23 — Optional anomaly detection

- Learn: event aggregation, time-window features, Isolation Forest output and evaluation.
- Practice: compare anomaly flags with rule-based alerts.
- Deliverable: optional anomaly-score field. An anomaly score is not an attack probability.

Day 24 — Optional phishing detector

- Learn: URL parsing, domains/subdomains, lexical features, keyword rules and classifier evaluation.
- Practice: use a synthetic benign/phishing URL dataset; treat length and symbols as weak signals only.
- Deliverable: optional phishing-risk indicator.

Day 25 — LLM basics

- Learn: LLM, prompts, structured output, context windows, hallucinations, local vs API models and API-key safety.
- Practice: generate a summary from a synthetic incident with events, evidence IDs, uncertainties and next steps.
- Deliverable: callable investigation-summary function.

Day 26 — Evidence-grounded AI

- Learn: retrieval/RAG, context selection, citation validation and prompt-injection risks in untrusted log content.
- Practice: send relevant event records and IDs; require evidence references and explicit uncertainty.
- Test: unsupported questions should get an 'insufficient evidence' answer.
- Deliverable: investigator assistant.

Day 27 — Streamlit dashboard

- Learn: Streamlit widgets, file upload, tables/charts, session state, downloads and error handling.
- Build: case overview, upload, event table, timeline, AI chat, hash and signature verification status.
- Deliverable: working dashboard.

Day 28 — PDF report and demo

- Build: PDF report using ReportLab with case ID, summary, timeline, alerts, evidence IDs/hashes, signature status, AI explanation, uncertainties and next steps.
- Rehearse: upload synthetic case → detect → timeline → ask AI → verify hash/signature → modify a copy → show mismatch → export PDF.
- Deliverable: prototype v1, README, synthetic dataset and presentation-ready demo.

Part 3 — Priorities, Resources and Demo Checklist

Priority levels

P1 — Must-have core

- Python log parser and synthetic dataset.
- Rule-based detection and incident timeline.
- SHA-256 verification.
- Streamlit dashboard.
- Evidence-linked AI summary.
- Automated tests for parsing, detection and integrity.

P2 — High-value extensions

- SQLite evidence vault.
- Conventional digital signatures.
- ML-DSA integration and verification.
- PDF reports and a more complete audit trail.

P3 — Optional

- Isolation Forest anomaly detection.
- Phishing-risk classifier.
- Docker/DVWA lab.
- Vector database, FastAPI, separate React frontend.

Free / official learning resources

Python Tutorial: https://docs.python.org/3/tutorial/ — Programming fundamentals and standard library.

TryHackMe: https://tryhackme.com/ — Beginner cybersecurity and networking labs; free access may vary.

MITRE ATT&CK;: https://attack.mitre.org/ — Tactics and techniques reference.

NIST PQC: https://csrc.nist.gov/projects/post-quantum-cryptography — Official post-quantum cryptography standards and project information.

Streamlit Docs: https://docs.streamlit.io/ — Build Python dashboards.

scikit-learn: https://scikit-learn.org/stable/ — Machine-learning concepts and tools.

ReportLab: https://docs.reportlab.com/ — PDF generation documentation.

Open Quantum Safe: https://openquantumsafe.org/ — PQC implementation ecosystem and compatibility notes.

Final end-to-end demo checklist

- Load a known synthetic case and show its source files.
- Show parsed events and detection alerts with evidence IDs.
- Explain the timeline as a possible sequence, not automatic proof of causation.
- Ask the AI a question and verify that claims cite actual event IDs.
- Verify a clean evidence file and signed manifest.
- Modify a copy and demonstrate hash/signature verification failure.
- Export and inspect the PDF report.
- Explain limitations: synthetic data, false positives, missing telemetry, AI uncertainty, and no automatic attribution.

Safety and integrity principles

- Use synthetic logs or data you are authorised to analyse.
- Do not expose intentionally vulnerable apps to the public internet.
- Do not send sensitive real evidence to an external LLM API without authorisation and a privacy review.

- Never claim a system is quantum-proof solely because one PQC algorithm is used.
- Keep private signing keys protected; a signature is only meaningful when key ownership and handling are trustworthy.

Project one-line pitch: QuantumShield AI is a student prototype for AI-assisted digital forensics that organises security events, builds evidence-linked incident timelines, helps investigators ask questions of case records, and demonstrates evidence-manifest integrity with hashing and post-quantum digital signatures.